ASSIGNMENT 2

NAME: JANANIPRIYA G
REG.NO: 192324041
SUBMISSION DATE:10/11/26
SLOT: A
SUBJECT:JAVA PROGRAMMING
COURSE CODE: CSA0903

Github Link :

Q1 – BUILDING TEXT UNDER LOAD: STRING, STRINGBUILDER OR STRINGBUFFER?

1.1 Comparison of String, StringBuilder and StringBuffer

- **String: Immutable. Every modification creates a new String object.**
- **StringBuilder: Mutable and faster for repeated text modifications.**
- **StringBuffer: Mutable and thread-safe because its methods are synchronized.**

For generating railway e-tickets, StringBuilder is suitable because it reduces unnecessary object creation.

1.2 Prediction of Performance

Class	Expected Performance	Reason
String	Slow	Creates new objects during concatenation
StringBuilder	Fast	Modifies the same object
StringBuffer	Moderate	Synchronization adds overhead

As the number of fields and tickets increases, StringBuilder is expected to perform better.

1.3 Experimental Workloads

Three workloads are considered:

1. **Small workload: 1 ticket with 5 fields.**

2. **Medium workload: 1,000 tickets with 20 fields each.**
3. **Large workload: 10,000 tickets with 50 fields each.**

Execution time can be measured using System.nanoTime(). The experiment should be repeated several times and the average time recorded.

1.4 Java Implementation

```
public class TicketDemo {
  public static void main(String[] args) {
    String ticket = "";
    StringBuilder sb = new StringBuilder();
    StringBuffer sf = new StringBuffer();

    for (int i = 1; i <= 5; i++) {
      ticket += "Field" + i + " ";
      sb.append("Field").append(i).append(" ");
      sf.append("Field").append(i).append(" ");
    }

    System.out.println("String: " + ticket);
    System.out.println("StringBuilder: " + sb);
    System.out.println("StringBuffer: " + sf);
  }
}
```

Output:

```
String: Field1 Field2 Field3 Field4 Field5
StringBuilder: Field1 Field2 Field3 Field4 Field5
StringBuffer: Field1 Field2 Field3 Field4 Field5
```

1.5 Design Decision

StringBuilder is recommended for generating individual e-tickets because it is fast and suitable for repeated modifications. StringBuffer is

recommended when multiple threads modify a shared booking log because it provides thread safety.

1.6 Alternative Concept and Complexity Analysis

StringJoiner, String.join() and String.format() can also be used to combine ticket fields.

Method	Time Complexity
Repeated String concatenation	$O(n^2)$
StringBuilder	O(n)
StringJoiner / String.join()	O(n)

Here, n represents the number of fields. Repeated String concatenation creates more intermediate objects, while StringBuilder reduces unnecessary object creation.

1.7 Conclusion

StringBuilder is the best choice for generating railway e-tickets because it provides better performance and uses memory efficiently. StringBuffer is suitable for shared text operations in a multithreaded environment.

Q2 – WHEN IS ARRAYLIST AN APPROPRIATE ENGINEERING CHOICE?

2.1 ArrayList and LinkedList Comparison

ArrayList and LinkedList are used to store and manage music playlists.

- **ArrayList: Uses a dynamic array and provides fast access to elements using an index.**
- **LinkedList: Uses connected nodes and supports efficient insertion and deletion at the beginning or end.**

Operation	ArrayList	LinkedList
Add at end	O(1) amortized	O(1)
Insert at front	O(n)	O(1)
Insert in middle	O(n)	O(n) to locate
Remove from front	O(n)	O(1)

Get kth song	**O(1)**	**O(n)**
Iteration	**O(n)**	**O(n)**

2.2 Testing Different Playlist Sizes

Three playlist sizes are considered:

1. **Small playlist – 100 songs.**
2. **Medium playlist – 1,000 songs.**
3. **Large playlist – 10,000 songs.**

The performance can be compared by testing insertion, deletion, searching and accessing songs.

2.3 When Is ArrayList Suitable?

ArrayList is suitable when:

- **Songs are frequently accessed by index.**
- **Songs are mainly added at the end.**
- **Iteration is performed frequently.**
- **Insertions and deletions in the middle are less frequent.**

LinkedList is more suitable when frequent insertions and deletions occur at the beginning or end.

2.4 Choosing a Queue for Up Next

For an Up Next playlist, songs may be added or removed from both ends. ArrayDeque is a suitable choice because it supports efficient insertion and deletion at both ends.

2.5 Complexity Comparison

Operation	**ArrayList**	**LinkedList**	**ArrayDeque**
Add at end	**O(1) amortized**	**O(1)**	**O(1) amortized**
Insert at front	**O(n)**	**O(1)**	**O(1) amortized**
Insert in middle	**O(n)**	**O(n) to locate**	**Not directly supported**
Get kth element	**O(1)**	**O(n)**	**O(n)**

Iteration	O(n)	O(n)	O(n)
Space per element	Array slot	Node with links	Array slot

2.6 Conclusion

ArrayList is appropriate for playlists that require frequent random access and iteration. LinkedList is useful for frequent insertions and deletions. ArrayDeque is preferred for an Up Next queue because it efficiently supports operations at both ends.

Q3 – ENGINEER THE hashCode–equals CONTRACT

3.1 Problem Analysis

In an event attendance system, duplicate student records must be identified using their registration number.

HashSet uses both hashCode() and equals() methods to identify duplicate objects. If two objects are equal, they must return the same hash code.

3.2 Problem with Missing hashCode()

If a Student class overrides only equals() and does not override hashCode(), HashSet may store duplicate student records because the objects can have different hash codes.

3.3 Correct Java Implementation

```
import java.util.*;


class Student {
  String regNo;
  String name;


  Student(String regNo, String name) {
    this.regNo = regNo;
    this.name = name;
  }
```

```
  public boolean equals(Object obj) {
    if (this == obj)
      return true;

    if (!(obj instanceof Student))
      return false;

    Student s = (Student) obj;
    return regNo.equals(s.regNo);
  }

  public int hashCode() {
    return Objects.hash(regNo);
  }
}

public class Main {
  public static void main(String[] args) {
    HashSet<Student> students = new HashSet<>();

    students.add(new Student("101", "Arun"));
    students.add(new Student("102", "Priya"));
    students.add(new Student("101", "Arun"));

    System.out.println("Unique students: " + students.size());
  }
}
```

Output:

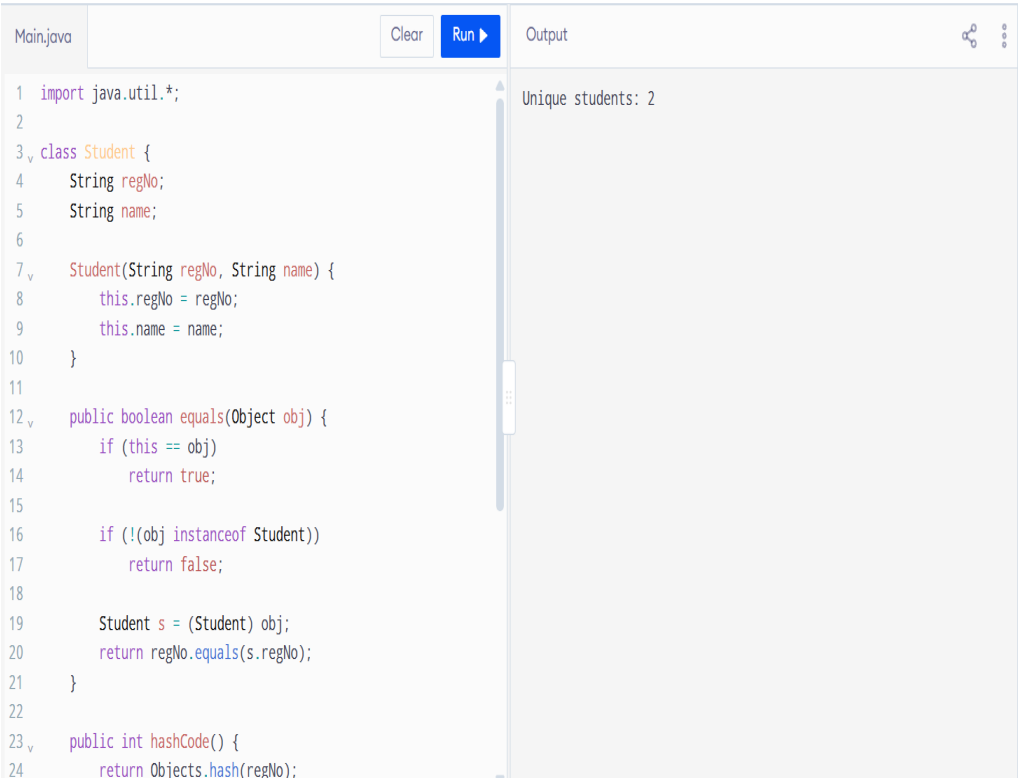
Unique students: 2

3.4 Test Cases

1. **Add two students with different registration numbers. Both should be stored.**
2. **Add two students with the same registration number. Only one should be stored.**
3. **Modify a registration number after inserting the student into the HashSet. This may cause incorrect lookup or removal because the object's hash code has changed.**

3.5 Alternative Approaches

- **TreeSet: Stores students in sorted order using a Comparator.**
- **HashMap: Uses registration numbers as keys to avoid duplicate records.**



3.6 Conclusion

The hashCode–equals contract is important for correctly identifying duplicate objects in HashSet and HashMap. Overriding both methods consistently ensures accurate duplicate detection and efficient collection operations.

4.1 Comparison of Map Implementations

A hospital pharmacy stores medicine details using medicine codes as keys.

- **HashMap: Stores key-value pairs and provides fast lookup but does not maintain order.**
- **TreeMap: Stores keys in sorted order.**
- **LinkedHashMap: Maintains the insertion order of elements.**

Operation	HashMap	TreeMap	LinkedHashMap
put()	O(1) average	O(log n)	O(1) average
get()	O(1) average	O(log n)	O(1) average
Iteration	O(n)	O(n)	O(n)
Ordering	No fixed order	Sorted order	Insertion order

4.2 Report Generation

Two reports are required:

1. **Medicine code order: TreeMap naturally produces medicines in sorted code order.**
2. **Insertion order: LinkedHashMap naturally maintains the order in which medicines were added.**

HashMap requires additional sorting or processing to generate ordered reports.

4.3 Testing Different Inventory Sizes

Three test sizes are considered:

- **Small inventory – 100 medicines.**
- **Medium inventory – 1,000 medicines.**
- **Large inventory – 10,000 medicines.**

Measure the time required for 100,000 lookups and report generation using System.nanoTime(). Verify that each map produces the required order.

4.4 Design Decision

HashMap is suitable for frequent medicine stock lookups because it provides average O(1) access.

TreeMap is suitable for generating reports in medicine-code order, while LinkedHashMap is suitable for maintaining insertion order.

For a system requiring all three operations, separate map structures can be maintained if the additional memory usage is acceptable.

4.5 Alternative Concept – Sorted Array

A sorted array can also store medicine codes and their stock details.

- **Searching: O(log n) using binary search.**
- **Insertion: O(n) because elements may need shifting.**
- **Code-order report: O(n).**
- **Space complexity: O(n).**

Structure	Lookup	Insertion	Ordered Report
HashMap	O(1) average	O(1) average	Requires sorting
TreeMap	O(log n)	O(log n)	Sorted order
LinkedHashMap	O(1) average	O(1) average	Insertion order
Sorted Array	O(log n)	O(n)	Sorted order

4.6 Conclusion

HashMap is recommended for frequent stock lookups, TreeMap for sorted medicine reports, and LinkedHashMap for insertion-order reports. The final choice depends on the pharmacy's lookup and reporting requirements.